

```
In [1]: # Import libraries
import nltk
import numpy as np
from sklearn.feature_extraction.text import CountVectorizer, TfidfVectorizer
from gensim.models import Word2Vec
```

```
In [2]: # Download tokenizer
nltk.download('punkt')

# Sample dataset
documents = [
    "Natural language processing is fun",
    "Machine learning is powerful",
    "Deep learning and NLP are related fields",
    "I love learning new things in AI"
]

print("Documents:")
for doc in documents:
    print("-", doc)

print("=*60)
```

```
Documents:
- Natural language processing is fun
- Machine learning is powerful
- Deep learning and NLP are related fields
- I love learning new things in AI
=====
```

```
[nltk_data] Downloading package punkt to
[nltk_data]   C:\Users\rushi\AppData\Roaming\nltk_data...
[nltk_data]   Package punkt is already up-to-date!
```

```
In [3]: # -----
# 1. Bag of Words (Count Occurrence)
# -----
count_vectorizer = CountVectorizer()
bow_matrix = count_vectorizer.fit_transform(documents)

print("\nBag of Words (Count Occurrence):")
print("Vocabulary:", count_vectorizer.get_feature_names_out())
print("Matrix:\n", bow_matrix.toarray())
```

```
Bag of Words (Count Occurrence):
Vocabulary: ['ai' 'and' 'are' 'deep' 'fields' 'fun' 'in' 'is' 'language' 'learn'
'love' 'machine' 'natural' 'new' 'nlp' 'powerful' 'processing' 'related'
'things']
Matrix:
[[0 0 0 0 0 1 0 1 1 0 0 0 1 0 0 0 1 0 0]
 [0 0 0 0 0 0 0 1 0 1 0 1 0 0 0 0 1 0 0]
 [0 1 1 1 1 0 0 0 0 1 0 0 0 0 1 0 0 1 0]
 [1 0 0 0 0 0 1 0 0 1 1 0 0 1 0 0 0 0 1]]
```

```
In [4]: # -----
# 2. Normalized Count (Term Frequency)
# -----
# Normalize by dividing each row by its total count
bow_array = bow_matrix.toarray()
normalized_bow = bow_array / bow_array.sum(axis=1, keepdims=True)
```

```
print("\nNormalized Count (Term Frequency):")
print(normalized_bow)
```

```
Normalized Count (Term Frequency):
[[0.      0.      0.      0.      0.      0.2
  0.      0.2      0.2      0.      0.      0.
  0.2      0.      0.      0.      0.2      0.
  0.      ]
 [0.      0.      0.      0.      0.      0.
  0.      0.25      0.      0.25      0.      0.25
  0.      0.      0.      0.25      0.      0.
  0.      ]
 [0.      0.14285714 0.14285714 0.14285714 0.14285714 0.
  0.      0.      0.      0.14285714 0.      0.
  0.      0.      0.14285714 0.      0.      0.14285714
  0.      ]
 [0.16666667 0.      0.      0.      0.      0.
  0.16666667 0.      0.      0.16666667 0.16666667 0.
  0.      0.16666667 0.      0.      0.      0.
  0.16666667]]
```

```
In [5]: # -----
# 3. TF-IDF
# -----
tfidf_vectorizer = TfidfVectorizer()
tfidf_matrix = tfidf_vectorizer.fit_transform(documents)

print("\nTF-IDF:")
print("Vocabulary:", tfidf_vectorizer.get_feature_names_out())
print("Matrix:\n", tfidf_matrix.toarray())
```

```
TF-IDF:
Vocabulary: ['ai' 'and' 'are' 'deep' 'fields' 'fun' 'in' 'is' 'language' 'lear
ning'
'love' 'machine' 'natural' 'new' 'nlp' 'powerful' 'processing' 'related'
'things']
Matrix:
[[0.      0.      0.      0.      0.      0.46516193
  0.      0.36673901 0.46516193 0.      0.      0.
  0.46516193 0.      0.      0.      0.46516193 0.
  0.      ]
 [0.      0.      0.      0.      0.      0.
  0.      0.4530051 0.      0.36674667 0.      0.57457953
  0.      0.      0.      0.57457953 0.      0.
  0.      ]
 [0.      0.39505606 0.39505606 0.39505606 0.39505606 0.
  0.      0.      0.      0.25215917 0.      0.
  0.      0.      0.39505606 0.      0.      0.39505606
  0.      ]
 [0.43003652 0.      0.      0.      0.      0.
  0.43003652 0.      0.      0.27448674 0.43003652 0.
  0.      0.43003652 0.      0.      0.      0.
  0.43003652]]
```

```
In [7]: # -----
# 4. Word2Vec Embeddings
# -----

# Tokenize sentences
tokenized_docs = [nltk.word_tokenize(doc.lower()) for doc in documents]
```

```

# Train Word2Vec model
w2v_model = Word2Vec(
    sentences=tokenized_docs,
    vector_size=50,      # size of word vectors
    window=3,           # context window
    min_count=1,         # include all words
    workers=4
)

print("\nWord2Vec Embeddings Example:")

# Get embedding for a word
word = "learning"
if word in w2v_model.wv:
    print(f"Vector for '{word}':\n", w2v_model.wv[word])
else:
    print(f"Word '{word}' not in vocabulary")

# Find similar words
print("\nWords similar to 'learning':")
print(w2v_model.wv.most_similar("learning"))

```

Word2Vec Embeddings Example:

Vector for 'learning':

```

[-1.0724545e-03  4.7286271e-04  1.0206699e-02  1.8018546e-02
-1.8605899e-02 -1.4233618e-02  1.2917745e-02  1.7945977e-02
-1.0030856e-02 -7.5267432e-03  1.4761009e-02 -3.0669428e-03
-9.0732267e-03  1.3108104e-02 -9.7203208e-03 -3.6320353e-03
 5.7531595e-03  1.9837476e-03 -1.6570430e-02 -1.8897636e-02
 1.4623532e-02  1.0140524e-02  1.3515387e-02  1.5257311e-03
 1.2701781e-02 -6.8107317e-03 -1.8928028e-03  1.1537147e-02
-1.5043275e-02 -7.8722071e-03 -1.5023164e-02 -1.8600845e-03
 1.9076237e-02 -1.4638334e-02 -4.6675373e-03 -3.8754821e-03
 1.6154874e-02 -1.1861792e-02  9.0324880e-05 -9.5074680e-03
-1.9207101e-02  1.0014586e-02 -1.7519170e-02 -8.7836506e-03
-7.0199967e-05 -5.9236289e-04 -1.5322480e-02  1.9229487e-02
 9.9641159e-03  1.8466286e-02]

```

Words similar to 'learning':

```

[('things', 0.2705654501914978), ('fields', 0.21057206392288208), ('nlp', 0.16
704177856445312), ('related', 0.1502026915550232), ('machine', 0.1320439279079
4373), ('ai', 0.1267007291316986), ('deep', 0.09984554350376129), ('love', 0.0
5936765298247337), ('new', 0.04979120194911957), ('is', 0.04237300902605057)]

```

In []: